EXPERT PARALLELISM ON A LAPTOP: RUNNING A 754B-PARAMETER MOE IN 24 GB

Haorui Li
Southeast University
haoruileee@gmail.com

https://github.com/haoruilee/DeepInfer-Mac

ABSTRACT

DeepSeek serves its 754B-parameter DeepSeek-V4.1-Flash model with expert parallelism: the 384 experts of each layer are spread over many GPUs, tokens are sent to the GPUs that hold their experts, and communication is overlapped with computation. We study what is left of this design on a single laptop with 24 GB of memory. Our system, DEEPINFER-MAC, keeps the dense weights (7.6 GB) in memory, stores the experts on the internal SSD, and after routing reads each missing expert with one zero-copy read into a preallocated GPU buffer. Decoding one token then becomes a chain: the router decides which experts to read only after attention, and the next layer waits for those experts. A two-parameter model of this chain fits measured decode speed within 2% across cache sizes. The chain explains why the cluster's overlap techniques help little here. Overlapping cached and missing experts gains 5%, and naive next-layer prefetching makes decode 5–11% slower. Using an interleaved ablation, we trace the loss to guesses that delay the reads a token is waiting for. We fix it by issuing guesses only after the current layer's reads have landed, a scheme we call idle-window speculation; inserting the guesses at the cold end of the cache adds a small gain that is within our confidence intervals. It makes decode 11–27% faster at every cache size we tested and does not change the output; in three traced tokens per setting, the SSD's busy time per step rises from 62–65% to 73–89%. On a MacBook Air, DEEPINFER-MAC decodes 1.23 tokens/s with an expert cache that keeps 9 GB free for macOS (1.35 tokens/s with a larger cache). Its next-token predictions agree with DeepSeek's API on 86.0% of open-ended continuation tokens (96.7% averaged over 17 cases that include 12 single-token retrieval checks), and it retrieves correctly at every tested length up to 16K tokens. We also show that, when the scales of each FP4 block product differ by at most 2^9, the sparse-attention indexer can use BF16 matrix units without changing its scores; with a fused kernel this makes an indexer query block about $2\times$ faster in long prefill (1.8–$2.1\times$ on three layers, $3.5\times$ on one).

1 INTRODUCTION

Mixture-of-experts (MoE) models activate only a small part of their parameters for each token (Fedus et al., 2022; DeepSeek-AI, 2024). DeepSeek-V4.1-Flash has 754B parameters in 40 layers, and each layer has 384 routed experts, of which a token uses six (DeepSeek-AI, 2026b). In the data center, DeepSeek serves this model with expert parallelism (EP) (DeepSeek-AI, 2026a). The experts of a layer are spread over many GPUs, and after routing each token is sent to the GPUs that hold its experts. The cost of this communication is hidden by overlapping two micro-batches, and frequently used experts are replicated.

This paper asks whether the same model can run on a MacBook Air with 24 GB of memory, and which of these techniques still help there. The converted model takes 338 GB, so it cannot be held in memory. However, the work for one token is much smaller. The dense layers take 7.6 GB at 8 bits per weight, and the six experts per layer that a token needs take about 3.3 GB in total. The laptop

also has a second device that can store experts: an SSD that reads about 3 GB/s. We therefore treat the SSD as the expert-holding device, the role that other GPUs play under EP.

We built DEEPINFER-MAC around this idea (Figure 1). After routing, the engine reads each missing expert from the SSD with one `preadv` call directly into a slot of a GPU buffer, without copies and without the OS page cache, and runs all experts of the layer in one gathered quantized matmul. Recently used experts stay in memory as a cache, which plays the role of expert replication.

Measuring this system showed where the time goes. A decoding token forms a strict chain in every layer: the router decides which experts to read only after attention, and the next layer can start only after the experts are computed. The SSD is therefore idle while the GPU works, and the GPU waits while the SSD reads. The cluster's micro-batch overlap helps little here: computing the cached experts while the missing ones load made decode 5% faster. Prefetching the next layer's experts (Eliseev & Mazur, 2023) should fill the idle time, but in our measurements it made decode 5–11% slower. An interleaved ablation shows why: the guesses compete with reads the token is waiting for. *Idle-window speculation* issues the guesses only after the current layer's misses have arrived, so that they use the SSD while the GPU computes; it also inserts them at the cold end of the cache, which adds a small gain within our confidence intervals. This makes decode 11–27% faster at every cache size we tested and leaves the output unchanged. In three traced tokens per setting, the SSD is busy 62–65% of a decode step without speculation and 73–89% with it.

The engine follows the official reference implementation op by op. It includes compressed sparse attention, Engram memory, Hyper-Connections (Zhu et al., 2025), the vision encoder, DSpark drafting and the FP8/FP4 roundings used in training, and our tests run the official PyTorch model as an oracle.

In summary, we make the following contributions:

- **Expert parallelism on one laptop.** We map EP to one GPU and one SSD: experts move to the GPU through zero-copy reads into fixed buffer slots, and an LRU cache replaces expert replication. We are not aware of another system that runs every component of DeepSeek-V4.1-Flash, including Engram, Hyper-Connections and vision, within 24 GB with parity tests against the reference (Sections 3 and 4).

- **A cost model and a study of parallelism.** We model a decode step as a chain of compute and SSD reads. A two-parameter fit matches measured decode speed within 2% across cache sizes; applied to the speculation runs it shows that speculative reads are not free, and traces show that they mostly fill time when the SSD was idle. We test parallelism at six levels and find which kinds help: those that give the SSD work while the chain leaves it idle, without taking memory or bandwidth from demand reads (Sections 3.2 and 6.2).

- **Idle-window speculation.** We find why next-layer expert prefetching (Eliseev & Mazur, 2023) slows decode on a memory-bound laptop: the guesses delay reads the token is waiting for. Issuing guesses only after the demand reads have landed, a known prefetch-control idea, turns the slowdown into a speed-up; an ablation separates it from cold insertion, which adds little (Section 3.3 and 6.2).

- **An exactly equal, faster indexer.** We prove that, when the scales of each FP4 block product differ by at most 2^9 the products stay in the normal FP32 range and the matrix units accumulate in FP32, the FP32 indexer score of DSA-style indexers can be computed with BF16 matrix units without changing a bit. Together with a fused head reduction, this makes the indexer about $2\times$ faster in long prefill (1.8–$2.1\times$ on three layers, $3.5\times$ on one). We also measure DoSA's exact screening (Meng & Fang, 2026) on this model's compressed two-level indexer, where it saves little at 16–32K keys (Section 5).

On a MacBook Air (M4, 24 GB), DEEPINFER-MAC decodes 1.23 tokens/s with a 5.5 GB expert cache, which keeps 9 GB free for macOS, and 1.35 tokens/s with 6.5 GB. Its next-token predictions agree with DeepSeek's API on 86.0% of open-ended continuation tokens, and it answers a retrieval question correctly at every tested length up to 16K tokens. The code, converter, tests and measurement scripts are open source.

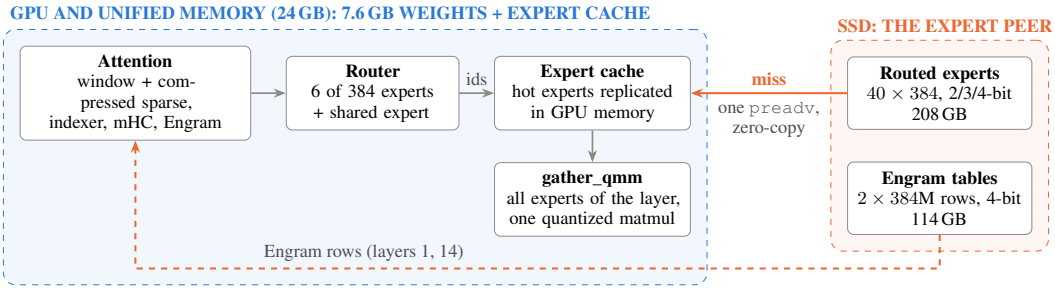


Figure 1: Overview of DEEPINFER-MAC. Dense weights and recently used experts are kept in unified memory. After routing, missing experts are read from the SSD directly into slots of preallocated GPU buffers. Within a layer, attention, router, reads and expert computation run one after another.

2 BACKGROUND AND MOTIVATION

2.1 DEEPSEEK-V4.1-FLASH

DeepSeek-V4.1-Flash has 754B parameters in 40 transformer layers with hidden size 5120 (DeepSeek-AI, 2026b). Each layer has an MoE block with one shared expert and 384 routed experts (intermediate size 2304); a sqrt-softplus router picks 6 of them for every token. Attention uses a 128-token sliding window plus *compressed sparse attention*. Layers 2–19 compress the keys at ratio 2 and layers 20–39 at ratio 1. A small multihead indexer (DeepSeek-AI, 2025) with 32 heads of dimension 128 then selects 512 compressed positions for each query. Selection has two levels: layer 20 picks 2048 candidate blocks of 8 positions, and later index layers only score inside those blocks. Layers 0 and 1 use the window only. Four layers (2, 8, 14 and 20) own the compressed KV caches and indexer keys, and the other layers reuse them. Layers 1 and 14 also contain an *Engram* module, which looks up embeddings of hashed n-grams ($n \leq 4$) in two tables with about 384M rows each. The residual stream uses Hyper-Connections with four streams and Sinkhorn-normalized mixing matrices (Zhu et al., 2025). The model also has three DSpark draft layers for speculative decoding and a 32-layer vision encoder.

The release stores dense weights in FP8 (E4M3 with 32×32 E8M0 block scales) and experts in FP4 (E2M1 with one E8M0 scale per 32 values), 510 GB in total. The reference inference code also rounds activations at fixed points during inference: FP8 before dense projections, FP4 for indexer queries and keys and for the compressed KV cache, and BF16 after each step of the indexer score. These roundings matter most where the model makes discrete choices. A score that rounds differently can change which keys the indexer selects or which experts the router picks.

2.2 EXPERT PARALLELISM IN THE DATA CENTER

DeepInfer (DeepSeek-AI, 2026a), DeepSeek's inference engine, serves this model on a GPU cluster. Attention runs with data and context parallelism, and the MoE blocks run with expert parallelism (EP). Under EP each GPU stores a subset of the experts. After routing, an all-to-all *dispatch* sends every token's hidden state to the GPUs that hold its experts, and a *combine* step sends the weighted outputs back. DeepSeek uses two more techniques to keep the GPUs busy (DeepSeek-AI, 2024). First, the computation of one micro-batch overlaps with the communication of another, so the all-to-all is hidden. Second, frequently used experts are replicated on several GPUs so that no single GPU becomes a bottleneck. Both techniques need many tokens in flight at the same time.

2.3 WHY A LAPTOP MIGHT BE ENOUGH

Our target is a MacBook Air with an M4 chip and 24 GB of unified memory, shared by the CPU and the GPU (about 120 GB/s). After conversion (Section 4) the model takes 338 GB, far more than the memory. However, a single token needs much less than that:

- The dense part (attention, shared experts, routers, mixers and Engram projections) takes 7.6 GB at 8 bits per weight.

Table 1: Expert parallelism (EP) on a GPU cluster and the corresponding mechanism in DEEPINFER-MAC.

	EP on a cluster	DEEPINFER-MAC
Where experts are stored	GPU memory of many devices	SSD records (208 GB)
After routing	dispatch tokens (all-to-all)	read missing experts (`preadv`, no copy)
Expert computation	grouped GEMM on each GPU	one `gather_qmm` over buffer slots
Hiding transfers	overlap two micro-batches	compute cached experts during reads
Load balancing	replicate hot experts	LRU expert cache in unified memory

- The routed experts that one token uses take about 3.3 GB: 40 layers times 6 experts, at 12–19 MB per expert after compression to 2–4 bits.

The internal SSD reads 2.6–3.0 GB/s. If the dense part stays in memory and about half of the needed experts are already cached, each token needs roughly 1.8 GB from the SSD. That is about 0.6 s of reading, so about one token per second looks possible.

This view treats the SSD as the device that holds the experts, the role that other GPUs play under EP. It raises the question this paper studies: *which parts of expert parallelism still help when there is one GPU, one SSD and one sequence?*

3 DESIGN

Figure 1 shows the design. Dense weights stay in memory. Experts stay on the SSD, and the engine reads the ones it needs into GPU buffers. Table 1 lists how each part of expert parallelism maps to our setting. We describe the expert path (§3.1) and its critical path (§3.2), then the two mechanisms that shorten it: idle-window speculation, which is new (§3.3), and an expert cache that plays the role of expert replication (§3.4).

3.1 READING EXPERTS INSTEAD OF SENDING TOKENS

Under EP, tokens travel to the experts. On a laptop the token is already next to the GPU, so we move the expert instead.

Records and slabs. We store each expert as one record on the SSD, aligned to 16 KiB. A record holds the packed weights, scales and biases of the expert's three projections, in the same layout that MLX (Hannun et al., 2023) uses for quantized tensors. For each precision layout, the engine allocates a *slab* at start-up: three GPU arrays of shape $[S, \cdot, \cdot]$, where slot s holds one expert. On a cache miss, the engine takes a free slot and issues a single `preadv` call whose buffers point into the three arrays at that slot. Because the CPU and GPU share memory, the GPU reads the data exactly where the SSD wrote it, with no extra copy. We open the files with `F_NOCACHE` so that macOS does not keep another copy in its page cache.

One matmul per projection. When the router of a layer finishes, the host reads the selected expert ids. This is the only CPU–GPU synchronization in a layer. The engine maps each expert to its slot and pins the slot until the layer is done. All (token, expert) pairs of the layer then run through one `gather_qmm` call per projection, indexed by slot, with SwiGLU and the routing weights fused into the same compiled function. This step plays the role of the combine step, without any communication.

3.2 THE CRITICAL PATH OF ONE TOKEN

A cluster serves many sequences, so while one micro-batch waits for the network another one computes. A laptop usually decodes one sequence, one token at a time. Figure 2 shows what happens in four consecutive layers. The GPU first runs attention, the mixers and the router. Only then does the host know which experts are needed, and the missing ones go to the SSD. Eight I/O threads issue the reads, which finish after 10–30 ms. The GPU then computes the experts, and the next layer can start only after that. As a result, the GPU and the SSD are rarely busy at the same time.

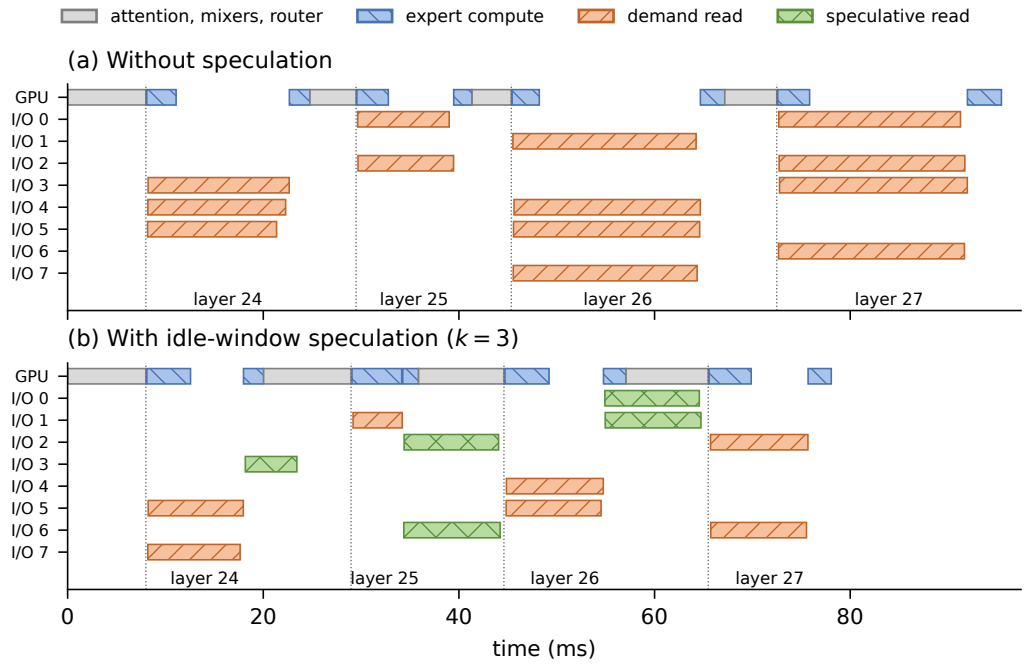


Figure 2: Measured timeline of four decode layers with a 5 GB expert cache. Gray: attention, mixers and router. Blue: expert computation (cached experts first, then the missing ones). Orange: demand reads, one row per I/O thread. Green: speculative reads (§3.3). (a) Reads start only after the router of their layer finishes, and the SSD is idle while the GPU works. (b) With idle-window speculation, guesses for the next layer fill this idle time; several experts of each layer are already in memory when the router finishes. Each panel shows the median of three traced tokens. To time GPU work we synchronize after each expert group, which makes a step 2% (a) and 14% (b) slower than untraced decoding in the same run.

This dependency gives a simple cost model. Let a be the time of attention, mixers and router in one layer, c the expert computation time, m_ℓ the number of missing experts in layer ℓ, b the size of one expert and B the SSD bandwidth. A decode step then takes about

$$t_{\text{token}} \approx \sum_{\ell=1}^{40} \left(a + c + \frac{m_\ell \, b}{B} \right). \tag{1}$$

On an idle machine we measured 0.80 s per token. The compute terms add up to 0.25 s. With a 49% hit rate the missing experts total 1.78 GB; the measured SSD wait was 0.55 s, which corresponds to 3.2 GB/s. These numbers roughly add up to the measured step, and they show three ways to make it shorter. The first is fewer misses, which needs a larger cache (§3.4). The second is to overlap the reads of a layer with the computation that does not depend on them; inside one layer, that is only the cached experts. The third is to read experts *before* the router asks for them, so that the SSD also works while the GPU computes (§3.3).

Section 6.2 measures each of these options, together with other kinds of parallelism from related systems.

3.3 Idle-window speculation

Equation (1) also shows when the SSD is idle: during a and c, while the GPU runs attention and computes experts. In a decode step this idle time adds up to about 0.25 s, enough to read about 50 experts. Reading the next layer's experts during this time would take reads off the critical path. Prefetching experts is not new (Eliseev & Mazur, 2023; Hwang et al., 2024; Xue et al., 2024a; Song et al., 2024a; Zhong et al., 2024). In particular, Eliseev & Mazur (2023) already guess the next

layer's experts by applying its gate to the current hidden state, and we use the same predictor. However, prefetching the most likely next-layer experts made decode 5–6% *slower* in our measurements. Looking at the trace, we suspected two causes, which the ablation in Section 6.2 later separated:

- **Guesses ran before needed reads.** The engine issued the guesses for layer $\ell + 1$ before the misses of layer ℓ. Both went to the same I/O queue, so every wrong guess delayed a read the token was waiting for.
- **Guesses might pollute the cache.** A guessed expert entered the LRU list as most recently used, so a wrong guess could push out an expert that had been used. The ablation shows that this effect is small at a 5 GB cache.

Idle-window speculation addresses both and needs no training or model change. Its essential part is the first: issue guesses only after the demand reads have landed. After the router of layer ℓ finishes, the engine also applies the router of layer $\ell + 1$ to the current hidden state, using its exact scoring (sqrt-softplus plus bias), and takes the top k experts as guesses. The guesses go to a separate thread that waits until all misses of layer ℓ have arrived and only then issues their reads. They therefore run while the GPU computes, in the window where the SSD would otherwise be idle. In addition, a guessed expert is inserted at the *cold* end of the LRU list, the end that is evicted first, similar to low-priority insertion in CPU caches (Qureshi et al., 2007). If layer $\ell + 1$ uses it, it moves to the hot end; if not, the next guess or miss replaces it before any used expert. A guess never evicts a slot that is pinned or in use by the current request. The change is about 40 lines in the expert store and the MoE block. The ablation in Section 6.2 shows that the first fix, waiting for the demand reads, provides most of the gain; cold insertion adds a little on top and hurts on its own. Figure 2b shows the effect: in the same four layers, guessed experts arrive during the GPU's work, and the demand reads that remain are fewer and shorter.

3.4 Replication: spending memory on the cache

Replicating hot experts is the part of EP that transfers most directly. A cluster copies busy experts so that no GPU waits; we keep recently used experts in memory so that no read is needed. Each hit removes one term b/B, about 5 ms, from Equation (1). We use least-recently-used replacement. On routing traces recorded from the real model, LRU had a higher hit rate than least-frequently-used at the same capacity, and capacity mattered more than the policy: 1.5 times more slots raised the hit rate from 0.38 to 0.47.

Because cache size drives decode speed, the rest of the design tries to save memory:

- Dense FP8 weights convert to MXFP8 without loss (Section 4), so we never keep a higher-precision copy.
- Expert reads skip the page cache, so the same expert is never stored twice.
- Speculative reads enter the cache cold (§3.3), and speculative decoding is off by default, so neither pushes useful experts out.

The cache size is set at start-up from the memory that is free at that moment, minus a 4 GB margin, and at least 9 GB always stay free for macOS. With less than that the system starts swapping and decode slows down sharply. On an idle 24 GB machine this gives 5–6 GB of expert slots.

We also provide an optional approximation based on the same idea. If a missing expert carries less than 8% of a token's routing weight, the engine skips it and saves the read. It lowers top-1 agreement with the API on open-ended text by 2.4 points. We did not measure its speed effect in an interleaved test, so we report no speed-up for it, and it is off by default.

4 Implementation

DeepInfer-Mac is about 3,800 lines of Python on MLX (Hannun et al., 2023), plus one Metal kernel (Section 5). We ported the official reference implementation op by op, and our tests run that implementation as an oracle. The engine is not lossless overall: experts are compressed to 2–4 bits (below). We wanted every speed-up to be checkable against the reference.

Table 2: Converted weights of DeepSeek-V4.1-Flash, as stored on disk (sizes in GiB; we write GB for GiB throughout).

Component	Format	Size	Location
Dense layers: attention, shared experts, routers, mHC, Engram projections	`mxfp8`, lossless	6.9	memory
Output head and other global tensors	affine 8-bit / BF16	0.7	memory
Embedding	BF16, memory-mapped	1.2	SSD
Routed experts, 40×384	affine 2/3/4-bit, $g = 64$	208.3	SSD
DSpark layers: dense parts / experts (3×128)	`mxfp8` / affine 2/3-bit	0.7 / 4.5	SSD
Vision encoder	BF16	0.9	SSD
Engram tables, 2×384M rows	affine 4-bit, $g = 32$	114.4	SSD
Total on disk		337.6	

Weight format. Table 2 shows where each part of the model is stored. The FP8 dense weights convert to MLX's `mxfp8` format without any loss: we keep the E4M3 values and copy each 32×32 E8M0 block scale to the per-32-element scales of MXFP8. Routed experts are converted from FP4 to 2–4-bit affine integers with group size 64. For each group we search for the clipping ratio that gives the smallest mean squared error. We chose bit widths per layer by comparing hidden states with converted and original experts (Appendix A). Lossless MXFP4 experts would need about 296 GB instead of 208 GB, which did not fit on our 1 TB disk next to the download. Engram tables are stored as 4-bit rows with group size 32 (160 bytes per row); the converted rows differ from the originals by 0.64% relative error.

One forward path. As in DeepInfer, prefill, decode, chunked prefill, prefix reuse and speculative verification all use the same forward function. It can process any chunk of tokens at any position. The per-sequence state has the sliding-window rings, the compressed KV and indexer-key caches and the compressor's unfinished tail. It can be saved and restored exactly. Two features use this. With *bounded replay*, layers 21–39 only process the last 140 prompt tokens, because they see older tokens only through layer 20's compressed cache and their own 128-token window. This is DeepInfer's default and roughly halves expert reads in long prefill. For *conversation reuse*, the engine keeps the last sequence and a snapshot taken one token before the end of the last prompt. A follow-up turn that extends either of them only processes the new tokens. The snapshot is needed in thinking mode, where the chat template removes the previous turn's reasoning from the next prompt.

Other components. Engram rows are read from the SSD tables through a row-level LRU cache; image tokens are left out of the n-gram hash, as in the reference. DSpark drafts are checked with speculative rejection sampling, which keeps the output distribution exact at any temperature. A small controller measures the throughput of speculative and plain steps and stops speculating when it is slower. The server supports the OpenAI, Anthropic and Responses APIs and reuses DeepSeek's prompt rendering and output parsing (DeepSeek-AI, 2026c). Structured outputs use XGrammar (Dong et al., 2024). The converter never stores the 510 GB release. It reads each tensor from the model hub with range requests, converts it on the GPU and writes it to its final place, and it can resume after any interruption.

Testing. The test suite runs DeepSeek's PyTorch model, with CPU versions of its TileLang kernels, on small random configurations that cover every mechanism. With activation rounding turned off on both sides, prefill and 19 decode steps reach a median logit cosine of at least 0.9999. The vision encoder matches the reference exactly in FP32. DSpark drafts equal the reference, and greedy speculative decoding gives the same tokens as greedy decoding. On the real checkpoint, every converted layer reaches a cosine of at least 0.99997 against the reference on a real chat prompt, and the Engram hash ids are identical.

5 Case study: the sparse-attention indexer

Long prompts on a laptop take minutes, and part of that time goes to the indexer. For a query with indexer heads $\mathbf{q}_1, \ldots, \mathbf{q}_H \in \mathbb{R}^d$ ($H = 32$, $d = 128$) and signed head weights $w_1, \ldots, w_H$, the

reference scores each visible compressed key $\mathbf{k}_j$ as

$$s_j \;=\; \mathrm{bf16}\Big(\sum_{h=1}^{H} \mathrm{bf16}\,\big(\,\big[\mathrm{bf16}(\mathbf{q}_h^\top \mathbf{k}_j)\big]_+ \, w_h\big)\Big), \tag{2}$$

summing in FP32, and keeps the $K = 512$ best keys. In layers 21–39, only keys inside the candidate blocks from layer 20 are eligible. Prefill scores every query against every visible key, so this cost grows with the square of the prompt length. We looked at two ways to make it cheaper: computing Equation (2) faster but exactly, and skipping keys that cannot win.

5.1 Faster scoring with the same results

BF16 products. The reference computes $\mathbf{q}_h^\top \mathbf{k}_j$ in FP32 and then rounds to BF16. Both inputs are FP4 values with power-of-two scales, which BF16 can represent exactly.

Lemma 1. *Let $\mathbf{q}, \mathbf{k} \in \mathbb{R}^{128}$ hold E2M1 values with one power-of-two scale per block of 32 elements, and let σ_b be the product of the two scales of block b. If $\max_b \sigma_b / \min_b \sigma_b \le 2^9$, then every partial sum of $\mathbf{q}^\top \mathbf{k}$, in any order, is exactly representable in FP32. A BF16 matrix multiplication with FP32 accumulation and one final rounding therefore returns $\mathrm{bf16}(\mathbf{q}^\top \mathbf{k})$, which is the reference result.*

Appendix B gives the proof. It assumes, besides the scale condition, that no block product leaves the normal FP32 range, and that the BF16 matrix multiplication accumulates in FP32; MLX's GEMM and GEMV kernels for BF16 inputs use FP32 accumulators, but we verified exactness empirically rather than for every kernel variant. If the condition does not hold, both versions are FP32 sums whose rounding can depend on the summation order, which is also true for different GPU kernels running the reference. On indexer inputs captured from a real 32K-token prompt, the BF16 version gave the same value for all 1,078M scores and ran the matmul 1.5–$1.9\times$ faster on the M4 GPU.

Fused head reduction. After the matmul, Equation (2) still needs a ReLU, a multiplication by w_h and a BF16 rounding per element, followed by a sum over heads. Written with array operations, this reads a $[\text{rows}, H, T]$ tensor three times; for 128 query rows and $T = 32$K keys that tensor has 134M elements. We replaced these operations with one Metal kernel. Each thread handles one (row, key) pair and loops over the heads, applying the same roundings in the same order. On all inputs we tested it gives the same results as the array form, and it is 5–$6\times$ faster for this step. Both sum the 32 heads in FP32, so their results could differ in the last bit if the summation orders differed and the terms spanned a very wide range; the reference's own reduction order is implementation-defined in the same way. Figure 3a shows the effect on real query blocks: the whole indexer block runs 1.8–$2.1\times$ faster on three layers and $3.5\times$ on one, and the selected keys are unchanged.

Fixed tie order. A partition-based Top-K leaves ties at the K-th place to the kernel. We sort by a 64-bit key built from the BF16 score and the position (Appendix C), so ties go to the earlier position; NaN scores are treated as $-\infty$. This makes the selection a pure function of the scores, which also matters for the exact screening below.

5.2 Skipping keys: exact screening

DoSA (Meng & Fang, 2026) skips keys while keeping the selection exact. A seed set of keys gives a lower bound τ on the K-th best score. A cheap upper bound U_j is then computed for every key, and keys with $U_j < \tau$ are dropped before full scoring. The DoSA paper reports indexer speed-ups of up to $2.74\times$ on GLM-5.2 and DeepSeek-V3.2, above $2\times$ at 512K tokens and more, with $K = 2048$. V4.1-Flash differs in three ways: it has fewer indexer heads, a smaller K, and a two-level candidate selection. We therefore measured how well the bound works on this model.

Setup. We saved the indexer inputs of a real 32K-token prompt at the eight index layers and study the seven that select keys by Equation (2) alone; layer 20, the candidate source, also needs every block maximum and cannot skip keys. We use 17 queries per layer: the last decode query and 16 queries spread over the last prefill block. For each query, the *eligible* keys are those it can see and, in layers 21–39, those inside its candidate blocks. τ is the K-th best exact score among the 4096

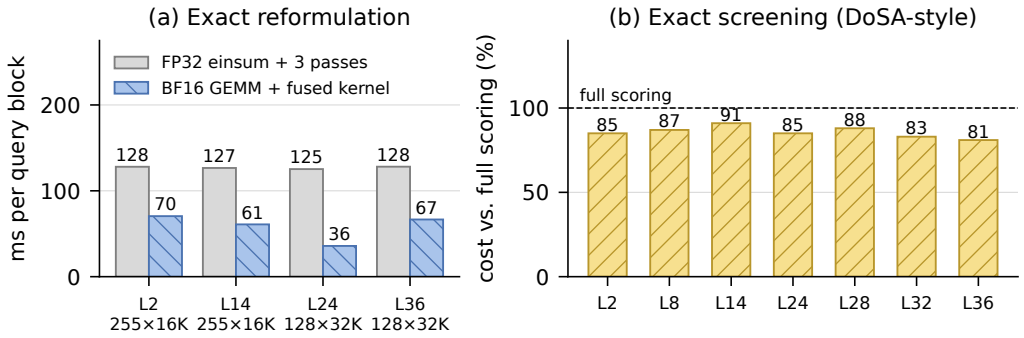


Figure 3: (a) Indexer time per prefill query block on real inputs (rows $\times$ keys), before and after the BF16 matmul and the fused head reduction. Scores and selected keys are identical on these blocks. (b) Cost of the screening bound (checked offline in float64 with slack for the BF16 roundings) relative to fully scoring the eligible keys, per index layer, at the best head budget ($K = 512$, 32K-token prompt, mean over 17 queries; includes scoring the seed keys).

most recent eligible keys (the seed set). As in DoSA, the bound evaluates the positive heads P and the e strongest negative heads E exactly and merges the other negative heads into one term:

$$U_j = \sum_{h \in P} w_h \left[\mathbf{q}_h^\top \mathbf{k}_j\right]_+ - \sum_{h \in E} |w_h| \left[\mathbf{q}_h^\top \mathbf{k}_j\right]_+ - \left[\textstyle\sum_{h \in N \setminus E} |w_h| \, \mathbf{q}_h^\top \mathbf{k}_j\right]_+ . \tag{3}$$

This is valid because $\sum_i [x_i]_+ \geq [\sum_i x_i]_+$. We enlarge U_j by 2^{-7} of the size of its terms to cover the BF16 roundings in Equation (2), and we check that no dropped key belongs to the true Top-K. The bound needs $m = |P| + |E| + 1$ dot products per key. A surviving key reuses the $|P| + |E|$ exact heads and only needs the remaining ones. Relative to fully scoring the eligible keys, the cost is the seed keys (scored fully), plus the other keys times m/H for the bound, plus the surviving keys times $|N \setminus E|/H$ for rescoring.

Results. Figure 3b and Table 7 show the cost at the best head budget for each layer, averaged over the 17 queries. A useful bound needs about 21–26 of the 32 heads, and the best total cost is 81–91% of full scoring. We also tried DoSA's joint bound for omitted positive heads, $\frac{1}{2}\big(\|\sum_h \mathbf{a}_h\|_2 + \sqrt{\sum_{i,j} |\mathbf{a}_i^\top \mathbf{a}_j|}\big)\,\|\mathbf{k}\|_2$, over all 128 dimensions and per 32-dimension group. With 8, 12 or 16 of the 21–23 positive heads kept, it did not drop any key that the candidate mask had not already removed. A low-rank bound from the leading singular vectors of the query heads also failed, because their singular values decay slowly. Two properties of V4.1 explain the gap to the DoSA setting:

- At 16–32K keys, $K = 512$ is a large fraction of the keys, so τ sits low in the score distribution.
- In layers 21–39 only the 16K positions in a query's candidate blocks are eligible, so the model's own two-level selection already removes many keys before the indexer score matters. (Our implementation still scores all keys and masks; computing only eligible keys would not help prefill either, because the candidate sets of 16 consecutive queries together cover 98% of the keys.)

In the best case screening would save 9–19% of indexer scoring, before the cost of compacting survivors on the GPU. We estimate the indexer at about 8% of a 32K-token prefill (from per-block timings), so the end-to-end gain would be about 1%. We left screening off and shared the measurements with the DoSA authors. Longer contexts may change the result, but a 128K-token prefill takes hours on this machine.

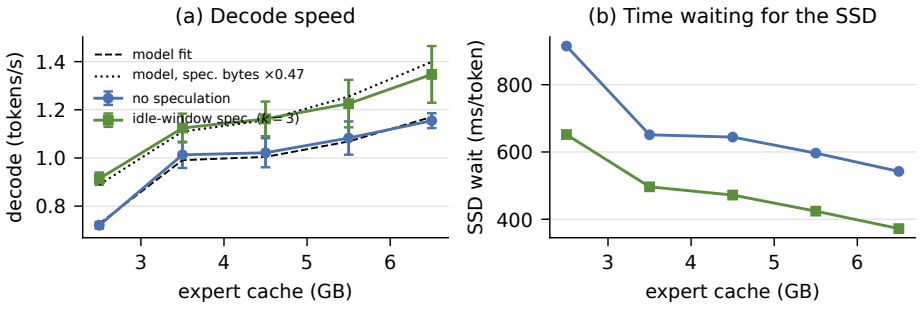


Figure 4: (a) Decode speed against expert cache size, with and without idle-window speculation (interleaved, 95% Student-t confidence intervals over 9 blocks), with the cost model fitted on the runs without speculation and the same model with the speculative bytes weighted by $\alpha = 0.47$. (b) Time per token spent waiting for the SSD.

6 Evaluation

Setup. All experiments run on a MacBook Air with an M4 chip (10-core GPU), 24 GB of memory and a 1 TB internal SSD, under macOS 26.6 and MLX 0.32, with other applications closed. For the main decode experiments (the ablation in Table 3 and the sweep in Figure 4), all compared configurations run interleaved in one process. Each *block* decodes 6 tokens with one configuration; the order of configurations rotates every round; we use three prompts (English prose, technical English and Chinese) and 3–4 rounds per prompt, and report the mean over blocks of the median time per token, with 95% confidence intervals from Student's t distribution. Speed-ups are paired: each block is compared with the block of the reference configuration from the same round. Other numbers come from earlier sessions and are marked as such. We answer three questions: how fast the system is and how well Equation (1) describes it (§6.1), which kinds of parallelism help (§6.2), and how close the output is to DeepSeek's own deployment (§6.3).

6.1 Decode speed and the cost model

Figure 4 shows decode speed for expert caches from 2.5 to 6.5 GB. Without speculation, speed rises from 0.72 to 1.16 tokens/s as the hit rate grows from 0.16 to 0.50. With idle-window speculation it rises from 0.91 to 1.35 tokens/s. The paired speed-up is 11–27% at every cache size (for example 1.112 ± 0.056 at 3.5 GB and 1.166 ± 0.096 at 6.5 GB), and speculation was faster in 43 of 45 blocks. The time spent waiting for the SSD drops by 24–31%. A cache of about 6 GB is the largest that leaves 9 GB for macOS on this machine.

We fitted Equation (1) in the form $t = C + D/B$, where D is the number of demand bytes per token, to the five runs without speculation ($C = 117$ ms, $B = 2.37$ GB/s; maximum error 2.0%). With five points this is a fit, not a prediction, and C and B should not be read as a compute time and a bandwidth on their own. As a held-out test we applied the same fit to the speculation runs. Counting only their demand bytes, it overestimates their speed: the predicted time per token is 23% too short on average, so the speculative reads are not free. Adding a term $\alpha S/B$ for the speculative bytes S and fitting α on the same runs gives $\alpha = 0.47$, with residuals from $+2.9\%$ to -6.3% that fall steadily with cache size; this is a description, not a held-out result. The traces (three tokens per setting, Figure 2) show the same effect directly. Without speculation the SSD is busy 62–65% of a decode step. With speculation it is busy 73–89%, and speculative reads that do not overlap any demand read take 28–34% of the step, i.e. they mostly use time in which the SSD was idle before.

Prefill. The first token arrives after about 10 s for a 14-token prompt, about 49 s for 1K tokens and about 9 minutes for 16K tokens (542 s in 4K-token chunks, 30 tokens/s). Even a short prompt routes to most experts of every layer, so the first chunk reads nearly the whole expert set once. Long prompts are limited by GPU compute instead: the MXFP8 projections take about 23% longer than a BF16 GEMM of the same shape, and bounded replay removes the decoder layers for all but the

Table 3: Decode schedule ablation at a 5 GB cache, all configurations interleaved (3 prompts × 4 rounds, 12 blocks of 6 tokens each). Speed-ups are paired against the serial schedule of the same round (mean and 95% confidence interval). Numbers are from the first ablation run; the matched-predictor rerun is reported in the text. All speculative variants use $k = 3$ guesses per layer; †ranks the guesses by the raw gate logits plus bias, the others by the router's exact scoring (see text).

Schedule	Guesses wait	Guesses cold	Tokens/s	Speed-up	SSD wait (ms/tok)
Serial (no overlap)	–	–	1.03	1.00	630
Overlap of cached and missing experts	–	–	1.07	1.05 ± 0.04	617
Naive next-layer prefetch (Eliseev & Mazur, 2023)†	no	no	0.97	0.95 ± 0.07	667
Guesses after demand submission	no	no	0.99	0.96 ± 0.05	637
Guesses, cold insertion only	no	yes	0.87	0.85 ± 0.06	770
Guesses, wait for demand reads only	yes	no	1.17	1.15 ± 0.05	464
Idle-window speculation	yes	yes	**1.20**	$\mathbf{1.17 \pm 0.05}$	**456**

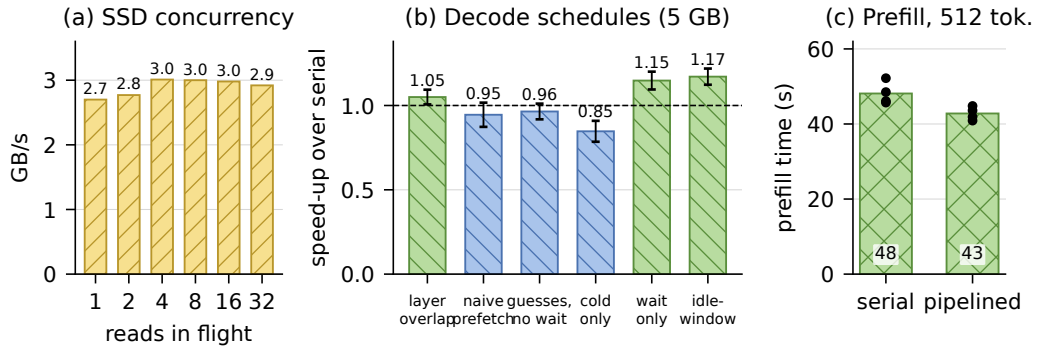


Figure 5: Parallelism on one laptop. (a) Read bandwidth for expert records against the number of reads in flight (`F_NOCACHE`, records not cached). (b) The schedule ablation of Table 3 (speed-up over the serial schedule, 95% confidence intervals); green: faster than serial. (c) Prefill of a 512-token prompt with and without reading the next expert batch while the current batch computes (bars: mean; dots: four runs each).

last 140 tokens. One 16K-token chunk is about as fast as four 4K chunks (518 s vs. 542 s) but needs 25.7 GB instead of 18.7 GB of MLX memory at peak, so we prefill in 4K chunks. Even a 4K chunk temporarily leaves less than the 9 GB reserve of Section 3.4, which is meant for decode, so long prompts run with other applications closed.

6.2 Where does parallelism help?

Table 3 and Figure 5 show the interleaved ablation at a 5 GB cache; Table 4 lists techniques measured in earlier sessions.

Inside the SSD. With one read in flight, expert records arrive at 2.70 GB/s; with four or more, at about 3.0 GB/s, after which the drive is saturated (Figure 5a). Splitting each expert into four smaller reads made no difference in an earlier session (1.11 vs. 1.12 tokens/s), because a 12–20 MB request is already large. The eight I/O threads collect this gain at no cost.

Inside a layer. Computing the cached experts while the missing ones load is the closest match to EP's micro-batch overlap. It made decode $5 \pm 4\%$ faster (8 of 12 blocks). Figure 2a shows why the gain is small: the cached experts of a layer compute in 2–4 ms, while its reads take 10–30 ms.

Across layers. Next-layer prefetching in the style of Eliseev & Mazur (2023) made decode 5% slower than serial and 10% slower than overlap alone. The ablation separates our two changes. Waiting for the demand reads before issuing guesses accounts for almost all of the gain ($1.15\times$). Cold insertion adds about 2 points on top, which is within the confidence interval at this cache size. On its own, cold insertion made decode 15% *slower*: guesses issued together with the demand reads

Table 4: Techniques measured in earlier sessions (tokens/s, against a baseline from the same session).

Technique	Baseline	With	Reason
4-way split reads per expert	1.12	1.11	one large read already saturates the SSD
DSpark, fixed draft length	1.28	0.35	verification reads the union of the drafts' experts
OS page cache behind a 5 GB slab	1.12	0.20–0.61	competes with the slab for memory

compete for the SSD, and cold entries are evicted again before the next layer can use them. With both changes, decode is 1.17 ± 0.05 times faster than serial and faster in 11 of 12 blocks. In this run the naive prefetcher ranks its guesses by the raw gate logits plus bias, while our guesses use the router's exact scoring. To rule out the predictor as the cause, we repeated the ablation with both using the exact scoring (a second interleaved run, 12 blocks). Naive prefetching was still slower than serial (0.89 ± 0.05, 0 of 12 blocks faster), although it obtained twice as many speculative hits per token (50 vs. 23); waiting for the demand reads gave 1.10 ± 0.05 and idle-window speculation 1.14 ± 0.05 (12 of 12 blocks faster). Guesses that compete with demand reads cost more than their hits save. In a separate, non-interleaved run on the same day (5 GB cache), three guesses per layer worked best (1.19 tokens/s, against 1.13 for two, 1.14 for four and 1.06 for six); with more guesses, they start to delay demand reads.

Across expert batches in prefill. During prefill a chunk of tokens goes through each layer, and the engine processes the layer's experts in batches of 16. Reading the next batch while the current one computes made a 512-token prefill 11% faster (42.8 s vs. 48.1 s, means of four runs; every pipelined run was faster than every serial run). In an earlier session with other programs using memory, the same comparison on a 1K-token prompt showed no difference (111 s vs. 111 s) and some pipelined runs swapped, because the extra batch needs memory.

Across tokens. Speculative decoding checks several drafted tokens in one pass (Leviathan et al., 2023). Here the drafted tokens route to different experts, so the check must read the union of their experts. With DSpark drafts of fixed length, decode dropped from 1.28 to 0.35 tokens/s in an earlier session. The controller from Section 4 detects this and falls back to plain decoding.

In the operating system. The page cache could act as a larger second-level cache behind the slab, but it competes with the slab for the same memory. With a 5 GB slab, enabling page caching lowered decode from 1.12 to 0.61 tokens/s (0.20 in a later warm run).

In summary, parallelism helps when it gives the SSD work during the time the dependency chain would leave it idle, without delaying the reads a token is waiting for or taking memory from the cache. Idle-window speculation meets these conditions; naive prefetching, speculative decoding and the page cache each violate one of them. The cache size still matters most: going from 2.5 to 6.5 GB raised decode speed by 47–60%.

6.3 Output quality

Against the official API. We use continuations sampled from DeepSeek's API for V4.1-Flash, published by Sanfilippo (2026), and run them through the converted model with teacher forcing. Figure 6 shows top-1 agreement for 17 cases. Five are open-ended continuations (prose, code, arithmetic and Italian). Twelve are needle-retrieval prompts whose lengths sit just below, at and above the sliding window (128), the compressed-key and Top-K boundaries (512 and 1024) and the candidate-block level (16K). The mean agreement per case is 96.7% and the mean NLL is 0.127, against 0.046 for the API's own model. The retrieval answer is correct at all twelve lengths, up to 16,385 tokens. Over the 129 tokens of the five open-ended continuations, agreement is 86.0%. We attribute the gap to the 2–3-bit experts, which matter more for open-ended text than for retrieval.

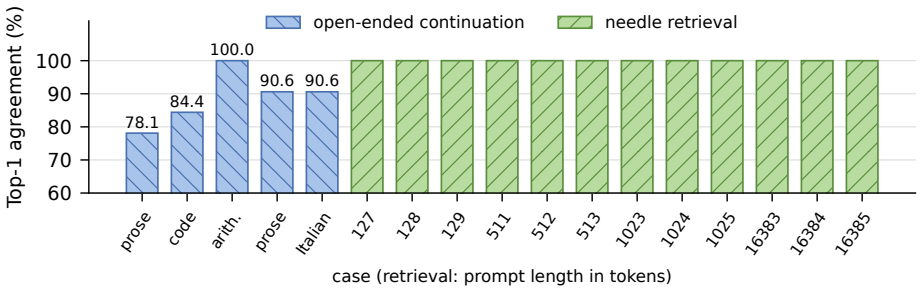


Figure 6: Top-1 agreement with continuations sampled from the official API. The retrieval prompts have one-token answers.

7 Related work

Running models larger than memory. FlexGen offloads weights, activations and the KV cache to CPU memory and disk and searches for a schedule that maximizes throughput for large batches (Sheng et al., 2023). LLM in a Flash keeps weights in flash storage and loads only the rows that predicted-active neurons need, reading them in large chunks (Alizadeh et al., 2024). PowerInfer keeps frequently activated neurons on the GPU and computes the rest on the CPU (Song et al., 2024b). For MoE models, Eliseev & Mazur (2023) combine an LRU expert cache with speculative prefetching of the next layer's experts on consumer GPUs; our predictor is theirs. Pre-gated MoE changes the model so that each layer selects the next layer's experts (Hwang et al., 2024), MoE-Infinity and ProMoE prefetch and cache experts from activation statistics (Xue et al., 2024a; Song et al., 2024a), AdapMoE adapts gating and prefetching to the cache (Zhong et al., 2024), Fiddler runs experts on the CPU instead of moving them (Kamahori et al., 2025), MoE-Lightning pipelines CPU and GPU work for large batches (Cao et al., 2025), and EdgeMoE and PowerInfer-2 bring expert or neuron offloading to phones (Yi et al., 2023; Xue et al., 2024b). llama.cpp memory-maps quantized weights and lets the operating system decide what stays in memory (Gerganov et al., 2023). Our setting differs in two ways. The dense part alone uses a third of the memory, and the workload is a single interactive sequence. Under these conditions, our measurements favour a fixed set of cache slots filled by reads that bypass the page cache over OS caching or cross-layer prefetching.

Expert parallelism. DeepSeek's serving stack spreads experts over many GPUs, overlaps all-to-all communication with computation using two micro-batches, and replicates heavily used experts (DeepSeek-AI, 2024; 2026a). We map these ideas to one GPU and one SSD and measure which of them still apply.

DeepSeek-V4 models on Apple silicon. Several community projects run DeepSeek-V4-generation models on Macs, and we learned from all of them. ds4 is a dedicated C and Metal engine whose API-sampled test vectors we use for evaluation (Sanfilippo, 2026). flash-moe streams experts from the SSD in C and Metal (danveloper, 2026). Two MLX ports target V4-Flash and V4.1-Flash (ssd-moe, 2026; PipeNetwork, 2026). DeepInfer-Mac follows the official reference implementation and its numerics op by op, which lets us test against it, and it shares its forward path and serving interface with DeepInfer (DeepSeek-AI, 2026a) and deepseek-recipe (DeepSeek-AI, 2026c).

Quantization. Post-training methods such as GPTQ lower weight precision with second-order error correction (Frantar et al., 2023). We use a simpler per-group clipping search because the converter streams each expert once from the network and has no calibration pass. We instead choose precision per layer and keep the dense weights lossless.

Sparse-attention indexers. DeepSeek Sparse Attention selects keys with a light multihead indexer (DeepSeek-AI, 2025). V4.1-Flash adds key compression and two-level candidate selection (DeepSeek-AI, 2026b), and IndexCache reuses indices across layers (Bai et al., 2026). Several methods speed up selection while keeping the full scorer's output. GVR checks a predicted thresh-

old during decoding (Cheng et al., 2026), LiteTopK fuses scoring with candidate filtering (Yin et al., 2026), and DoSA drops keys using score bounds (Meng & Fang, 2026), following the MaxScore and WAND idea from document retrieval (Turtle & Flood, 1995; Broder et al., 2003). MISA screens with a subset of heads but does not guarantee the full scorer's selection (Zhou et al., 2026).

Prefetch control. Giving demand requests priority over prefetches and inserting prefetched data with low priority are established techniques in processor memory systems (Lee et al., 2008; Srinath et al., 2007; Qureshi et al., 2007). Idle-window speculation applies both to expert reads from an SSD. Our contribution there is the measurement that shows why naive expert prefetching loses on a memory-bound laptop and that these two known fixes turn the loss into a gain, not the fixes themselves.

8 LIMITATIONS AND CONCLUSION

Limitations. At about 1–1.35 tokens/s, DEEPINFER-MAC is suitable for interactive use with short answers and for offline jobs, not for serving many users. It runs one sequence at a time, and long prompts take minutes (about 9 minutes for 16K tokens). The 2–3-bit experts lower agreement with the API on open-ended text to 86.0% of tokens, although retrieval is not affected; machines with more disk can store lossless MXFP4 experts with a one-line change. All results come from one machine, with short prompts and short decode blocks; we did not measure decode at long context. We did not compare against other engines: the community engines we cite run related checkpoints with different quantization on other machines, and porting them to this setting was out of scope. A Mac with a faster SSD or more memory would shift the balance between the techniques we measured. Our screening study covers 16–32K keys and does not settle the question for contexts of 128K tokens and more.

Conclusion. We asked what expert parallelism becomes on a laptop where the only other device is an SSD. Reading experts into fixed GPU buffers without copies replaces the token dispatch of a cluster, and an LRU cache replaces expert replication. A simple fit of the resulting chain describes decode speed from the bytes read per token, and traces show the SSD idle for a third of each step. The cluster's overlap techniques recover little of that time, and naive prefetching makes it worse because its guesses delay the reads a token is waiting for. Issuing the guesses only after those reads land, idle-window speculation, fills the idle time and makes decode 11–27% faster. With these pieces, a 24 GB MacBook Air runs the 754B-parameter DeepSeek-V4.1-Flash at 1.23 tokens/s with a cache that keeps 9 GB free for macOS, with outputs close to DeepSeek's own deployment. We release the engine, the converter, the tests and the measurement scripts.

AI USE STATEMENT

We used generative AI tools for drafting code, experiments and this manuscript, which were reviewed and revised by the author. The author takes responsibility for the final manuscript and implementation.

REPRODUCIBILITY STATEMENT

The engine, converter and tests are released under the Apache 2.0 license at `https://github.com/haoruilee/DeepInfer-Mac`. Section 4 and Appendix A describe the weight format, and `python -m deepinfer_mac convert` reproduces the checkpoint from the public release. `tests/` runs the parity tests against the official reference implementation when its source is available (set `DSV41_REF`); otherwise those tests are skipped. The measurements use these scripts; their raw outputs (logs and JSON files) are included in `paper/logs`. One interleaved ablation run (`ablation2.json`) was disturbed by an external event, with blocks below 0.1 tokens/s in every configuration; we repeated it (`ablation3.json`) and do not use it. The scripts are:

- `scripts/spec_ablation.py` and `scripts/spec_sweep.py`: the interleaved schedule ablation (Table 3) and the cache-size sweep (Figure 4), with raw per-token times and store counters;
- `scripts/trace_decode.py`: the per-layer timelines (Figure 2);
- `scripts/bench_io_parallel.py` and `scripts/prefill_overlap.py`: SSD concurrency and prefill pipelining (Figure 5);
- `scripts/decode_speed.py`, `scripts/spec_ab.py`, `scripts/schedule_ab.py`: earlier decode measurements;
- `scripts/trace_policies.py`: replacement-policy replay on recorded routing traces;
- `scripts/score_api.py`: agreement with the API-sampled vectors of Sanfilippo (2026); `scripts/real_check.py`: per-layer parity on the real checkpoint; `scripts/quant_damage.py`: per-layer effect of expert compression;
- `scripts/capture_indexer.py` and `scripts/dosa_study.py`: the indexer study (the captured inputs, about 300 MB, are not included); `scripts/bench_kernels.py`: the indexer and MXFP8 kernel timings.

`paper/figures/make_plots.py` draws the figures; values it does not read from `paper/figures/data` are copied from the logs named above. The numbers in Section 6.3 come from the v0.1.1 build (`final_eval.log`); a later build scored 97.1% mean agreement (`after_score.log`).

REFERENCES

Keivan Alizadeh, Iman Mirzadeh, Dmitry Belenko, Karen Khatamifard, Minsik Cho, Carlo C. Del Mundo, Mohammad Rastegari, and Mehrdad Farajtabar. LLM in a flash: Efficient large language model inference with limited memory. In *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics*, 2024. URL https://arxiv.org/abs/2312.11514.

Yushi Bai, Qian Dong, Ting Jiang, Xin Lv, Zhengxiao Du, Aohan Zeng, Jie Tang, and Juanzi Li. IndexCache: Accelerating Sparse Attention via Cross-Layer Index Reuse. *arXiv preprint arXiv:2603.12201*, 2026. URL https://arxiv.org/abs/2603.12201.

Andrei Z. Broder, David Carmel, Michael Herscovici, Aya Soffer, and Jason Zien. Efficient query evaluation using a two-level retrieval process. In *Proceedings of the Twelfth International Conference on Information and Knowledge Management*, pp. 426–434. ACM, 2003. doi: 10.1145/956863.956944.

Shiyi Cao, Shu Liu, Tyler Griggs, Peter Schafhalter, Xiaoxuan Liu, Ying Sheng, Joseph E. Gonzalez, Matei Zaharia, and Ion Stoica. MoE-Lightning: High-throughput MoE inference on memory-constrained GPUs. In *Proceedings of the 30th ACM International Conference on Architectural Support for Programming Languages and Operating Systems*, 2025.

Long Cheng, Ritchie Zhao, Timmy Liu, Mindy Li, Xianjie Qiao, Kefeng Duan, Yu-Jung Chen, Xiaoming Chen, Bita Darvish Rouhani, and June Yang. Guess-Verify-Refine: Data-Aware Top-K for Sparse-Attention Decoding on Blackwell via Temporal Correlation. *arXiv preprint arXiv:2604.22312*, 2026. URL https://arxiv.org/abs/2604.22312.

danveloper. flash-moe. https://github.com/danveloper/flash-moe, 2026.

DeepSeek-AI. DeepSeek-V3 technical report. *arXiv preprint arXiv:2412.19437*, 2024. URL https://arxiv.org/abs/2412.19437.

DeepSeek-AI. DeepSeek-V3.2: Pushing the Frontier of Open Large Language Models. *arXiv preprint arXiv:2512.02556*, 2025. URL https://arxiv.org/abs/2512.02556.

DeepSeek-AI. DeepInfer. https://github.com/deepseek-ai/DeepInfer, 2026a.

DeepSeek-AI. DeepSeek-V4.1-Flash: Pushing the limits of KV cache compression. Technical report, DeepSeek-AI, 2026b. URL https://huggingface.co/deepseek-ai/DeepSeek-V4.1-Flash/blob/main/DeepSeek_V41_Tech_Report.pdf.

DeepSeek-AI. deepseek-recipe. https://github.com/deepseek-ai/deepseek-recipe, 2026c.

Yixin Dong, Charlie F. Ruan, Yaxing Cai, Ruihang Lai, Ziyi Xu, Yilong Zhao, and Tianqi Chen. XGrammar: Flexible and efficient structured generation engine for large language models. *arXiv preprint arXiv:2411.15100*, 2024. URL https://arxiv.org/abs/2411.15100.

Artyom Eliseev and Denis Mazur. Fast inference of mixture-of-experts language models with offloading. *arXiv preprint arXiv:2312.17238*, 2023. URL https://arxiv.org/abs/2312.17238.

William Fedus, Barret Zoph, and Noam Shazeer. Switch transformers: Scaling to trillion parameter models with simple and efficient sparsity. *Journal of Machine Learning Research*, 23(120):1–39, 2022. URL https://www.jmlr.org/papers/v23/21-0998.html.

Elias Frantar, Saleh Ashkboos, Torsten Hoefler, and Dan Alistarh. GPTQ: Accurate post-training quantization for generative pre-trained transformers. In *International Conference on Learning Representations*, 2023. URL https://arxiv.org/abs/2210.17323.

Georgi Gerganov et al. llama.cpp. https://github.com/ggml-org/llama.cpp, 2023.

Awni Hannun, Jagrit Digani, Angelos Katharopoulos, and Ronan Collobert. MLX: Efficient and flexible machine learning on Apple silicon. https://github.com/ml-explore/mlx, 2023.

Ranggi Hwang, Jianyu Wei, Shijie Cao, Changho Hwang, Xiaohu Tang, Ting Cao, and Mao Yang. Pre-gated MoE: An algorithm-system co-design for fast and scalable mixture-of-expert inference. In *Proceedings of the 51st Annual International Symposium on Computer Architecture*, 2024. URL https://arxiv.org/abs/2308.12066.

Keisuke Kamahori, Tian Tang, Yile Gu, Kan Zhu, and Baris Kasikci. Fiddler: CPU-GPU orchestration for fast inference of mixture-of-experts models. In *International Conference on Learning Representations*, 2025.

Chang Joo Lee, Onur Mutlu, Veynu Narasiman, and Yale N. Patt. Prefetch-aware DRAM controllers. In *Proceedings of the 41st Annual IEEE/ACM International Symposium on Microarchitecture*, 2008.

Yaniv Leviathan, Matan Kalman, and Yossi Matias. Fast inference from transformers via speculative decoding. In *International Conference on Machine Learning*, 2023. URL https://arxiv.org/abs/2211.17192.

Fanxu Meng and Taosong Fang. DoSA: Dots Sparse Attention. *arXiv preprint arXiv:2610.13990*, 2026. URL https://arxiv.org/abs/2610.13990.

PipeNetwork. deepseek-v41-mlx. https://github.com/PipeNetwork/deepseek-v41-mlx, 2026.

Moinuddin K. Qureshi, Aamer Jaleel, Yale N. Patt, Simon C. Steely, and Joel Emer. Adaptive insertion policies for high performance caching. In *Proceedings of the 34th Annual International Symposium on Computer Architecture*, pp. 381–391, 2007.

Salvatore Sanfilippo. ds4. https://github.com/antirez/ds4, 2026.

Ying Sheng, Lianmin Zheng, Binhang Yuan, Zhuohan Li, Max Ryabinin, Beidi Chen, Percy Liang, Christopher Ré, Ion Stoica, and Ce Zhang. FlexGen: High-throughput generative inference of large language models with a single GPU. In *International Conference on Machine Learning*, 2023. URL https://arxiv.org/abs/2303.06865.

Xiaoniu Song, Zihang Zhong, Rong Chen, and Haibo Chen. ProMoE: Fast MoE-based LLM serving using proactive caching. *arXiv preprint arXiv:2410.22134*, 2024a.

Yixin Song, Zeyu Mi, Haotong Xie, and Haibo Chen. PowerInfer: Fast large language model serving with a consumer-grade GPU. In *Proceedings of the ACM SIGOPS 30th Symposium on Operating Systems Principles*, 2024b. URL https://arxiv.org/abs/2312.12456.

Santhosh Srinath, Onur Mutlu, Hyesoon Kim, and Yale N. Patt. Feedback directed prefetching: Improving the performance and bandwidth-efficiency of hardware prefetchers. In *Proceedings of the 13th International Symposium on High Performance Computer Architecture*, 2007.

ssd-moe. deepseek-v4-flash-mlx. https://github.com/ssd-moe/deepseek-v4-flash-mlx, 2026.

Howard Turtle and James Flood. Query evaluation: Strategies and optimizations. *Information Processing & Management*, 31(6):831–850, 1995. doi: 10.1016/0306-4573(95)00020-H.

Leyang Xue, Yao Fu, Zhan Lu, Luo Mai, and Mahesh Marina. MoE-Infinity: Offloading-efficient MoE model serving. *arXiv preprint arXiv:2401.14361*, 2024a. URL https://arxiv.org/abs/2401.14361.

Zhenliang Xue, Yixin Song, Zeyu Mi, Le Chen, Yubin Xia, and Haibo Chen. PowerInfer-2: Fast large language model inference on a smartphone. *arXiv preprint arXiv:2406.06282*, 2024b.

Rongjie Yi, Liwei Guo, Shiyun Wei, Ao Zhou, Shangguang Wang, and Mengwei Xu. EdgeMoE: Fast on-device inference of MoE-based large language models. *arXiv preprint arXiv:2308.14352*, 2023.

Ziqi Yin, Jianyang Gao, Peiqi Yin, Jiangneng Li, and Gao Cong. LiteTopK: Exploiting the Curse of Dimensionality for a Fused Indexer-TopK Kernel in Long-Context Sparse Attention. *arXiv preprint arXiv:2607.11976*, 2026. URL https://arxiv.org/abs/2607.11976.

Shuzhang Zhong, Ling Liang, Yuan Wang, Runsheng Wang, Ru Huang, and Meng Li. AdapMoE: Adaptive sensitivity-based expert gating and management for efficient MoE inference. In *Proceedings of the IEEE/ACM International Conference on Computer-Aided Design*, 2024.

Ruijie Zhou, Fanxu Meng, Yufei Xu, Tongxuan Liu, Guangming Lu, Muhan Zhang, and Wenjie Pei. MISA: Mixture of Indexer Sparse Attention for Long-Context LLM Inference. *arXiv preprint arXiv:2605.07363*, 2026. URL https://arxiv.org/abs/2605.07363.

Defa Zhu, Hongzhi Huang, Zihao Huang, Yutao Zeng, Yunyao Mao, Banggu Wu, Qiyang Min, and Xun Zhou. Hyper-connections. In *International Conference on Learning Representations*, 2025. URL https://arxiv.org/abs/2409.19606.

A WEIGHT LAYOUT

Each routed expert is one record of three projections: gate W_1 and up W_3 (2304×5120) and down W_2 (5120×2304). Each projection is stored as packed integers, per-group scales and per-group biases in MLX's affine format with group size 64, and the record is padded to a multiple of 16 KiB. Table 5 gives the bit widths per layer. We chose them with `scripts/quant_damage.py`, which runs real layers once with the converted experts and once with the original FP4 experts and compares hidden states layer by layer. The first four layers, the layers that own KV caches or index keys, and the decoder band received 3 bits, the last layer 4 bits, and the rest 2-bit gate/up projections. The DSpark draft experts use 2-bit gate/up and 3-bit down projections.

Table 5: Expert bit widths (W_1/W_3/W_2) and record sizes.

Layers	Bits	Record
0–3, 8, 14, 20–38	3/3/3	15.5 MB
4–7, 9–13, 15–19	2/2/3	12.5 MB
39	4/4/4	19.9 MB

B PROOF OF LEMMA 1

E2M1 magnitudes are $\{0, 0.5, 1, 1.5, 2, 3, 4, 6\}$, i.e., integers in $\{0, \dots, 12\}$ times 2^{-1}. In block b with scales $s_{q,b}$ and $s_{k,b}$, each product $q_i k_i$ is therefore an integer of magnitude at most 144 times $\frac{1}{4}\sigma_b$, where $\sigma_b = s_{q,b}s_{k,b}$ is a power of two. Let $\sigma_{\min}$ and $\sigma_{\max}$ be the smallest and largest block products. Every product, and hence every partial sum over any subset of the 128 terms, is an integer multiple of $\frac{1}{4}\sigma_{\min}$. Its magnitude is at most $128 \cdot 144 \cdot \frac{1}{4}\sigma_{\max} < 2^{15} \cdot \frac{1}{4}\sigma_{\max}$. Expressed in units of $\frac{1}{4}\sigma_{\min}$, a partial sum is an integer below $2^{15}\,\sigma_{\max}/\sigma_{\min} \le 2^{24}$, which FP32's 24-bit significand represents exactly. Hence every FP32 addition in any order is exact, the accumulated value equals the real dot product, and the single final rounding to BF16 equals $\mathrm{bf16}(\mathbf{q}^\top \mathbf{k})$. This assumes that the block products stay in the normal FP32 range, i.e. $\frac{1}{4}\sigma_{\min} \ge 2^{-126}$ and no overflow. The quantizer clamps each scale at 2^{-126}, so the assumption can fail for blocks of near-zero activations; on our captured inputs the scale exponents lie roughly between -8 and 4. □

C TOP-K ORDER KEY

For a BF16 score with bit pattern b (16 bits), the key $\kappa(b) = 2^{16} - 1 - b$ if the sign bit is set and $b + 2^{15}$ otherwise is monotone in the score value, after mapping -0 to $+0$. For position $j < 2^{31}$ we select the K largest values of $\kappa(b) \cdot 2^{32} + (2^{31} - 1 - j)$, a 64-bit integer, with a partition. This yields the order "higher score first, then earlier position" regardless of the partition kernel. In the candidate-block level the query's own block has score $+\infty$, which maps to the largest key after NaNs are replaced by $-\infty$, so it is always kept.

D PER-CASE API AGREEMENT

Table 6: Teacher-forced evaluation against continuations sampled from the official API. NLL is the mean negative log-likelihood per continuation token under each model.

Case	Prompt	Continuation	NLL (ours)	NLL (API)	Top-1 agreement
Prose	15	32	0.733	0.038	78.1%
Code	17	32	0.637	0.278	84.4%
Arithmetic	18	1	0.000	0.000	100%
Prose	21	32	0.244	0.115	90.6%
Italian	15	32	0.512	0.354	90.6%
Retrieval (×12)	127–16,385	1	≤0.006	0.000	100% each
Mean over 17 cases			0.127	0.046	96.7%

E SCREENING STUDY DETAILS

Table 7 lists, for each index layer, the best configuration of the bound averaged over 17 queries (the last decode query and 16 prefill queries), with a seed set of the 4096 most recent eligible keys. e is the number of negative heads evaluated exactly, m the resulting number of dot products per key (positive heads, e negative heads and the merged term; it varies slightly between queries), and the cost includes scoring the seed keys. `scripts/dosa_study.py` reproduces the table and checks that no dropped key belongs to the true Top-K.

Table 7: Best screening configuration per layer (32K-token prompt; layers 2–14 see 16K compressed keys, layers 24–36 see 32K keys, of which about 16K are eligible). Survive is the fraction of the non-seed eligible keys whose bound is not below τ.

Layer	e	m (mean)	Survive	Cost vs. full scoring
2	0	24	18.1%	85%
8	0	23	31.7%	87%
14	4	26	24.4%	91%
24	4	21	38.5%	85%
28	0	22	40.0%	88%
32	0	24	8.9%	83%
36	0	22	19.9%	81%